Document de conception :

Application Android de création et résolution de puzzles à partir d'images

UE : Développement d'applications mobiles

Étudiant : CHERRAK Adham Salah
N° étudiant : 32221997
Formation : Licence 3 Informatique

Date de rendu : 20/022026

Université de Lorraine
Faculté des sciences et technologies Année universitaire : 2025–2026 **SOMMAIRE :**

1. Introduction
2. Contexte, objectifs et périmètre
3. Choix d'architecture
4. Modèle de données (UML)
5. Spécification des activités (pages)
6. Navigation et interactions
7. Cas d'utilisation
8. Règles métier et algorithmes
9. Persistance des données
10. Aspects techniques Android
11. Plan de développement
12. Stratégie de tests et validation
13. Conclusion
14. Annexes

1. Introduction

Ce document présente la conception d'une application Android permettant à un utilisateur de créer des puzzles à partir de ses propres photos puis de les résoudre de manière interactive.
L'objectif est de proposer une solution fonctionnelle, modulaire et évolutive, respectant les contraintes du sujet : création de puzzle, jeu, sauvegarde/chargement, et qualité d'ergonomie.

Le document décrit :

- Le modèle de données,
- L'architecture logicielle,
- Les écrans de l'application,
- Les règles métier,
- La stratégie de développement et de validation.

2. Contexte, objectifs et périmètre

2.1 Contexte

Le projet est réalisé individuellement et versionné sur GitLab. L'évaluation s'appuie sur :

1. Le document de conception,
2. La démonstration de l'application sur appareil mobile.

2.2 Objectifs

L'application doit permettre :

- La **création** d'un puzzle à partir d'une image choisie par l'utilisateur ;
- La **résolution** du puzzle via des interactions de manipulation de pièces.

2.3 Périmètre fonctionnel

2.3.1 Fonctionnalités obligatoires — Construction

- Création d'un puzzle (nom),
- Choix d'image (galerie/caméra),
- Définition d'un nombre de pièces approximatif,
- Déduction lignes/colonnes,

- Type de découpage (droit, polygonal, arrondi),
- Sauvegarde et chargement d’un puzzle existant.

2.3.2 Fonctionnalités obligatoires — Jeu

- Chargement d’un puzzle,
- Sauvegarde/reprise d’une partie,
- Parcours des pièces libres,
- Rotation gauche/droite,
- Placement sur grille avec validation,
- Retrait d’une pièce placée,
- Zoom sur la zone de jeu,
- Aperçu du modèle correspondant.

2.3.3 Bonus

- Découpe plus complexe,
- Amélioration UX/visuel,
- Options avancées.

3. Choix d’architecture

3.1 Pattern architectural: MVP (Model-View-Presenter) :

Le MVC classique est peu adapté à Android car les Activity mélangent trop facilement la logique métier et l'affichage, ce qui nuit à la maintenabilité. Le MVVM avec LiveData/ViewModel est plus moderne mais introduit une complexité (Data Binding, observateurs) qui n'est pas justifiée pour un projet individuel de cette taille. Le MVP offre un bon compromis : séparation claire des responsabilités, testabilité, sans surcharge technique.

Rôles des trois couches :

- **Model** : les entités de données (Puzzle, Piece, Partie, Placement) et les classes de persistance (lecture/écriture JSON, accès fichiers). Cette couche est indépendante d'Android.
- **View** : les Activity et leurs layouts XML. Elles n'ont aucune logique métier — elles délèguent tout au Presenter et se contentent d'afficher ce qu'on leur demande.
- **Presenter** : contient toute la logique applicative (calcul de la grille, validation d'un placement, gestion de la rotation, progression...). Il reçoit les événements de la View et met à jour le Model.

3.2 Organisations des packages :

```
com.example.puzzle/
├── model/          → Puzzle, Piece, Partie, Placement, TypeDecoupage
├── view/           → AccueilActivity, CreationPuzzleActivity,
│                     JeuPuzzleActivity, ListePuzzlesActivity, PartiesActivity
├── presenter/      → CreationPuzzlePresenter, JeuPuzzlePresenter, ...
├── repository/     → PuzzleRepository, PartieRepository (accès fichiers JSON)
├── utils/          → ImageUtils, GridUtils, RotationUtils
└── cutting/        → DroitCutter, PolygonalCutter, ArrondiCutter
```

3.3 **Gestion des images**

Un point technique critique concerne la gestion mémoire des images. L'application distingue deux usages :

- **Affichage** : une version redimensionnée (échantillonnée via BitmapFactory.Options.inSampleSize) est utilisée pour l'affichage à l'écran, afin d'éviter les OutOfMemoryError.
- **Génération des pièces** : la source pleine résolution est utilisée uniquement lors de la découpe initiale du puzzle, puis les pièces découpées sont sauvegardées séparément.

3.4 Stratégie de découpage des pièces

Pour garder le code organisé et évolutif, chaque type de découpage est géré par une classe séparée. Toutes ces classes partagent le même fonctionnement de base : elles reçoivent l'image source et les informations de la grille, et retournent la pièce découpée. Les trois types prévus sont :

- **DroitCutter** : découpe rectangulaire simple,
- **PolygonalCutter** : pièces avec encoches à angles droits,
- **ArrondiCutter** : pièces avec encoches courbes, comme un puzzle classique.

Cette organisation permet d'ajouter un nouveau style de découpage facilement, sans toucher au reste de l'application.

4. Modèle de données (UML)

4.1 Entités principales

Puzzle

- identifiant, nom, chemin image,
- dimensions image,
- nombre de lignes/colonnes, • type de découpage, • liste de pièces.

Piece

- identifiant,
- cellule cible (ligne/colonne),
- rotation courante et cible,
- état placé/non placé,
- coordonnées courantes (mode libre).

Partie

- identifiant, référence puzzle,
- dates (création, dernière sauvegarde),
- pièce courante,
- placements validés, • progression.

Placement

- identifiant de pièce,
- cellule placée, • rotation,
- validité.

ParametresPuzzle

- nombre de pièces approximatif,
- type de découpage,
- ratio cible.

4.2 Cardinalités

- Un Puzzle contient plusieurs Piece.
- Une Partie est associée à un seul Puzzle.
- Une Partie contient plusieurs Placement.
- Un Placement cible une Piece.

5. Spécification des activités (pages)

5.1 AccueilActivity

Rôle : point d'entrée.

Actions : Créer puzzle, Charger puzzle, Reprendre partie.

5.2 CreationPuzzleActivity Rôle :

saisie des paramètres de création.

Composants :

- champ nom,
- boutons galerie/caméra,
- sélecteur nb pièces,

- sélecteur type découpage,
- bouton générer/sauvegarder.

5.3 ListePuzzlesActivity

Rôle : lister les puzzles existants.

Actions : jouer, modifier, supprimer.

5.4 JeuPuzzleActivity Rôle :

interface de résolution.

Composants :

- zone puzzle (zoom/pan),
- pièce courante,
- boutons précédent/suivant,
- rotation gauche/droite,
- placer, retirer, sauvegarder, • aperçu image modèle.

5.5 PartiesActivity

Rôle : reprendre une partie sauvegardée.

Contenu : liste avec progression et date

6. Navigation et interactions

6.1 Graphe de navigation

- Accueil → Création
- Accueil → Liste puzzles
- Accueil → Parties
- Création → Jeu
- Liste puzzles → Jeu
- Parties → Jeu

6.2 Règles d'interaction

- Confirmation de suppression d'un puzzle,
- Sauvegarde avant sortie d'une partie,
- Message utilisateur en cas d'erreur de placement ou de permission refusée.

7. Cas d'utilisation :

7.1 UC-01 — Créer un puzzle

Acteur : utilisateur
Préconditions : permissions accordées **Scénario nominal :**

1. Ouvre l'écran de création ;
2. Saisit le nom ;
3. Choisit une image ;
4. Fixe les paramètres ;
5. Valide ;
6. L'application génère les pièces ;
7. Sauvegarde le puzzle et confirme.

Postcondition : puzzle disponible en liste.

7.2 UC-02 — Jouer une partie

1. Chargement d'un puzzle ;
2. Sélection de la pièce courante ;
3. Rotation éventuelle ;
4. Placement sur cellule ;
5. Validation ;
6. Répétition jusqu'à complétion.
    i. **Extension :** placement invalide → message + aucun enregistrement.

7.3 UC-03 — Sauvegarder / Reprendre

1. Sauvegarde explicite ou automatique ;
2. persistance JSON de l'état ;
3. reprise ultérieure fidèle de la progression.

8. Règles métier et algorithmes

8.1 Déduction lignes/colonnes

8.1 Déduction lignes/colonnes À

partir de :

- N = nb pièces approximatif,
- W/H = ratio image.

Principe

- rows ≈ sqrt(N / ratio),
- cols ≈ N / rows,
- ajustement pour obtenir une grille proche de N avec des pièces de ratio proche de 1

8.2 Rotation

- rotation gauche : (r + 270) % 360
- rotation droite : (r + 90) % 360

8.3 Validation de placement

Une pièce est valide si :

- Cellule proposée = cellule cible,
- Rotation courante = rotation cible.

8.4 Progression (en pourcentage)

Progression = (nombre_pieces_placees / nombre_total_pieces) * 100

9. Persistance des données

```
/fichier
/puzzles/{puzzleId}/ ○
    image.jpg
    ○ puzzle.json
/parties/ ○
    Partie.json
```

9.2 Contenu sauvegardé

Puzzle

- Métadonnées (nom, dimensions, découpage, grille). **Partie**
- Placements validés,
- Pièce courante,
- Progression,
- Date de dernière sauvegarde.

9.3 Stratégie de robustesse

- Écriture atomique (temp puis rename),
- Validation JSON au chargement,
- Gestion des erreurs I/O avec message utilisateur

10. Aspects techniques Android

10.1 Permissions

- Médias (selon API Android),
- Caméra

10.2 Performance image

- Chargement échantillonné pour affichage,
- Conservation d'une source de bonne qualité pour génération.

10.3 Compatibilité

- Gestion différenciée des permissions selon niveau d'API

11. Plan de développement

Itération 1 — Fondations

- Modèle de données,
- Création puzzle (galerie),
- Découpage droit,
- Sauvegarde/chargement puzzle.

Itération 2 — Cœur du jeu

- Écran de jeu,
- Parcours pièces,
- Rotation, placement, retrait,
- Progression

Itération 3 — Persistance partie + UX

- Sauvegarde/reprise partie,
- Zoom/pan,
- Aperçu modèle, • Feedback placement.

12. Stratégie de tests et validation

12.1 Tests fonctionnels

- Création puzzle complète,
- Chargement puzzle existant,
- Placement valide/invalide, • Reprise exacte après sauvegarde.

12.2 Critères d'acceptation

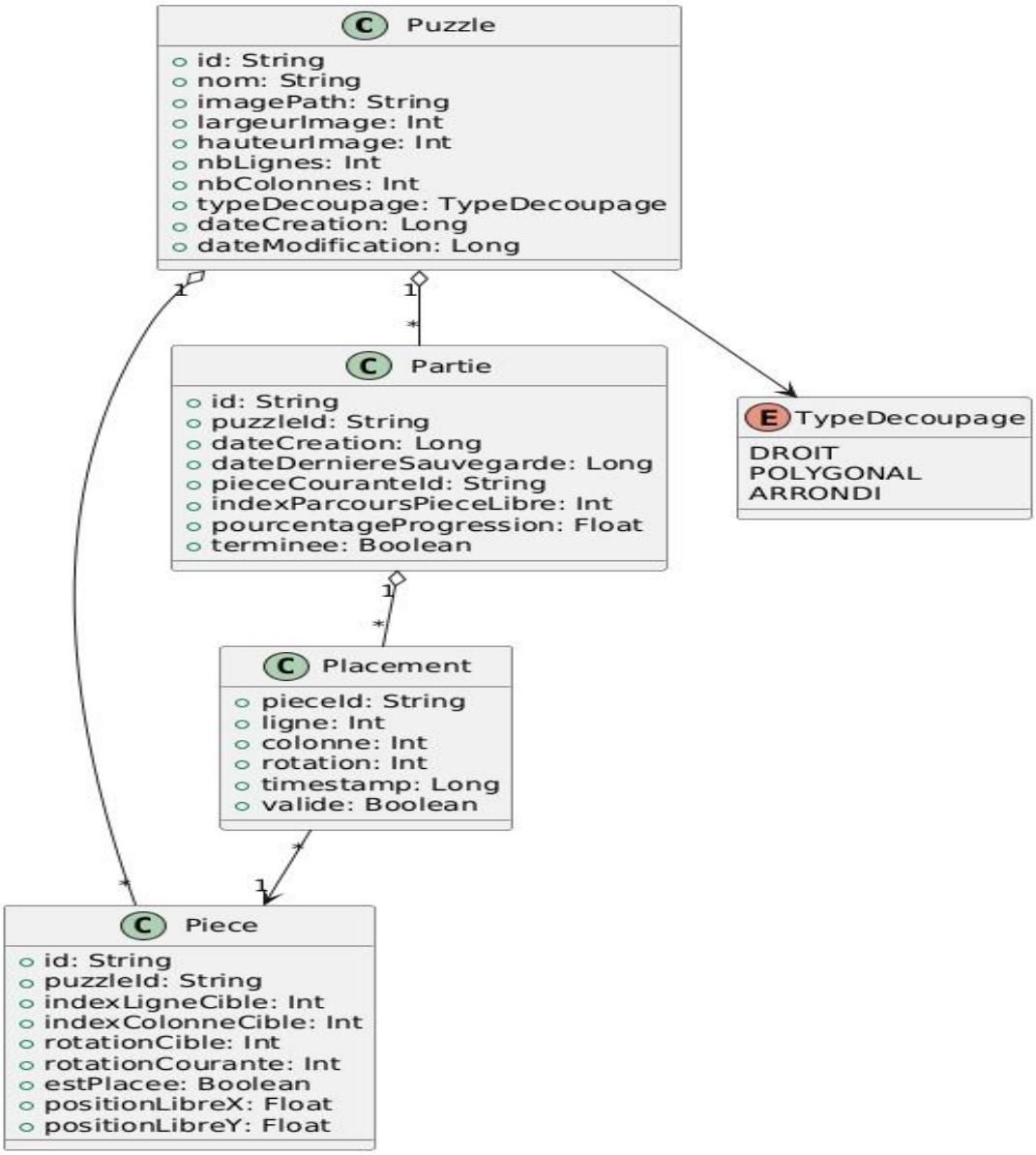
- Toutes fonctionnalités obligatoires opérationnelles,
- Application stable en démonstration, • Cohérence entre code, UX et document.

Conclusion

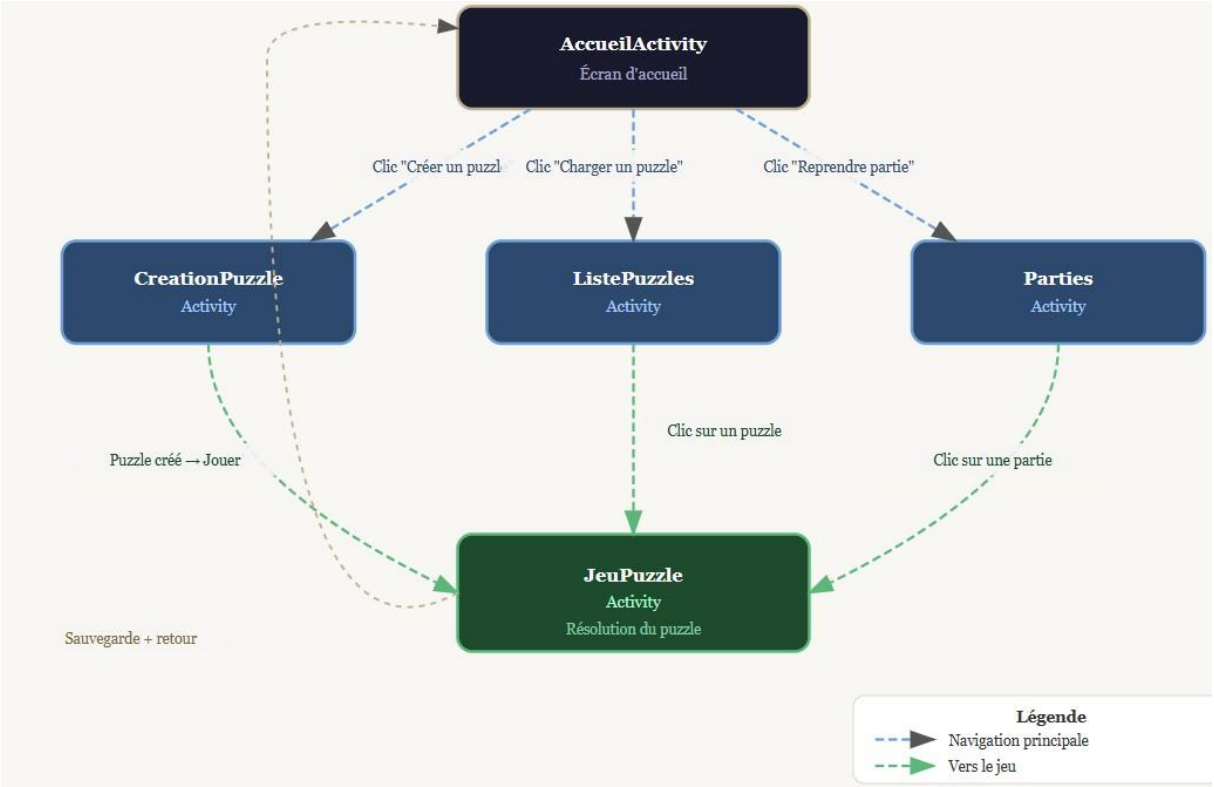
La conception proposée répond aux exigences du sujet avec une architecture claire, un modèle de données cohérent et un plan de réalisation progressif.
La stratégie retenue sécurise une version fonctionnelle complète (MVP) avant intégration des options bonus, maximisant la qualité du livrable final et la réussite de la démonstration.

Annexe :

Digaramme de classe



Shema de navigation :



Quelques maquettes (de presentation) :